

CS 159 Week 7:

Vector Semantics

Acknowledgements:

Portions of these slides were contributed by, and/or based on work from, Justine Zhang and Jack Hessel



(<https://xkcd.com/1838>)

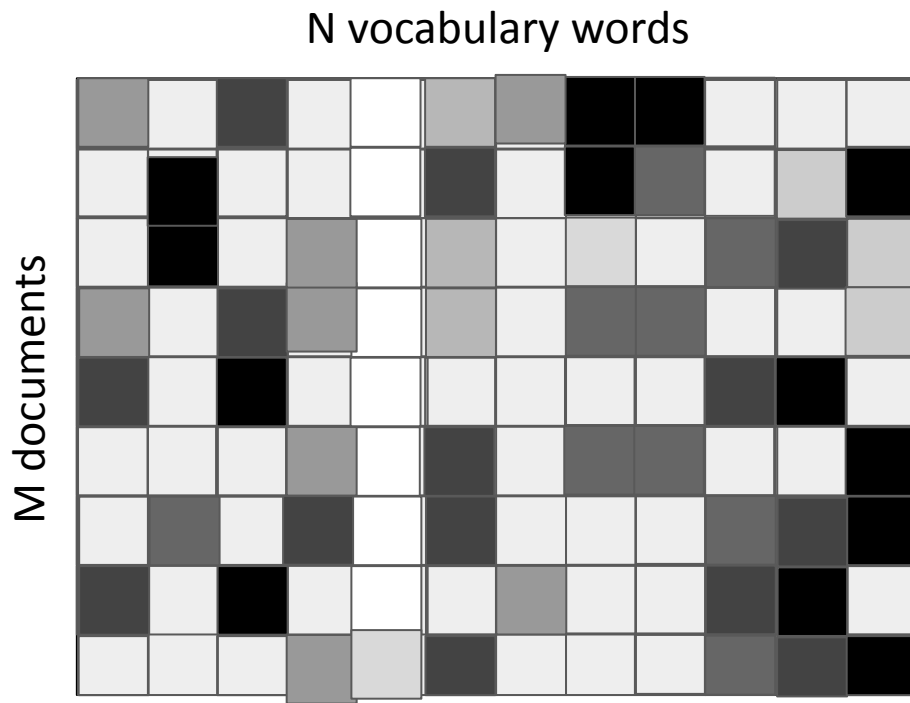
The rest of October at a glance

- This Thursday: **Lab 5** released (the last lab!); due October 24 (the Friday before the midterm)
- Sometime this week: **midterm details** released on course website (come to class Thursday if you have questions!)
- No class October 14 (**Fall break!**)
- October 16 (Thursday after break): **special wrap-up lecture** connecting course topics to LLMs
- Week of October 21: NLP ethics + midterm prep
- Week of October 28: Midterm! (take-home)
- Week of November 4: Prof Chang out for NLP conference (with class activities that week related to the conference!)

Last week...

Term-document matrix: a matrix of per-document word frequencies (counts)

$T =$



Last week...

Term-document matrix: a matrix of per-document word frequencies (counts)

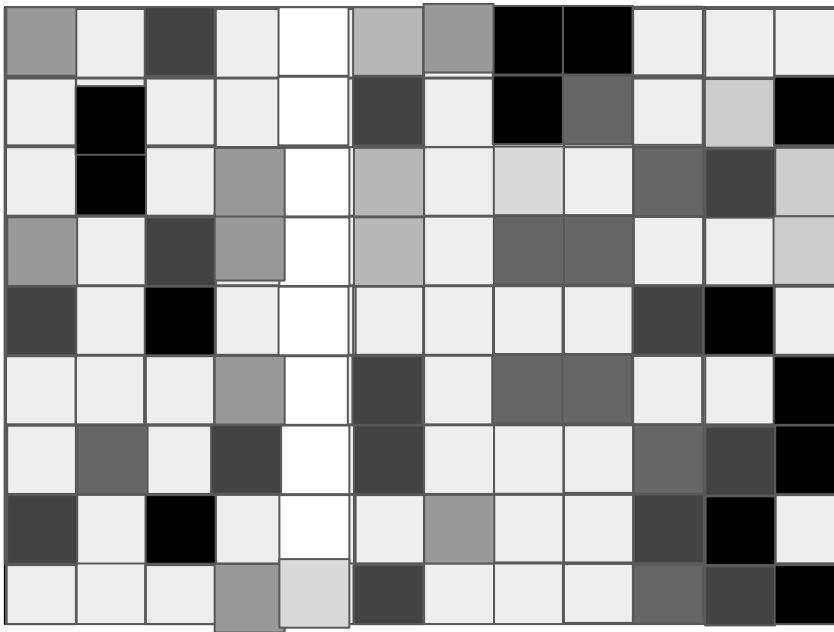
$$T =$$

Each matrix element is
computed as:

$T[i,j] = C_i(j)$ = the count of word
 j in document i

M documents

N vocabulary words

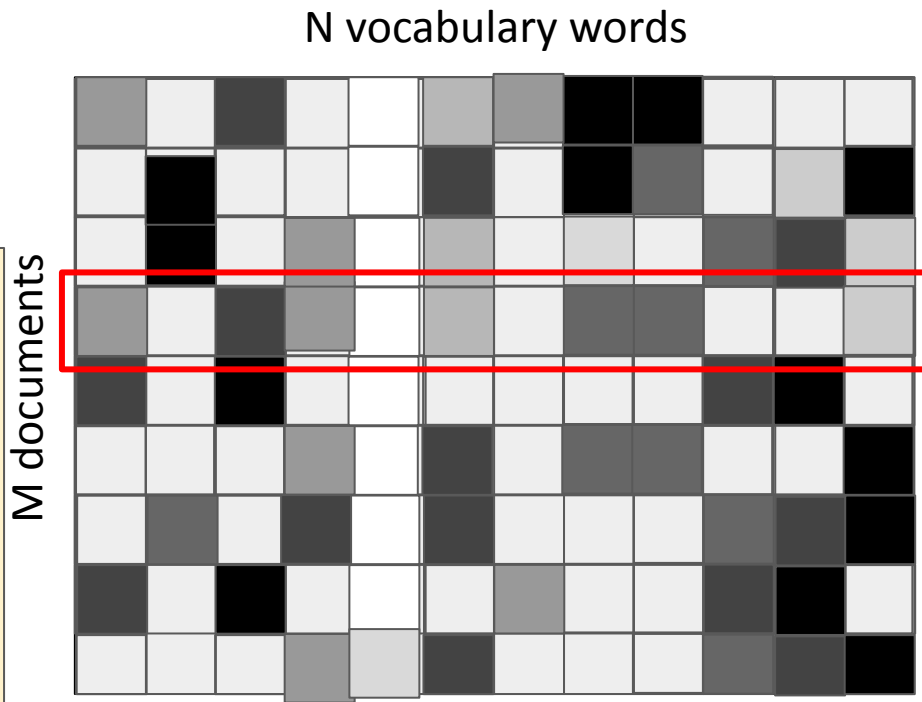


Last week...

Term-document matrix: a matrix of per-document word frequencies (counts)

$$T =$$

The i th row of T is a **vector representation** of the i th document.

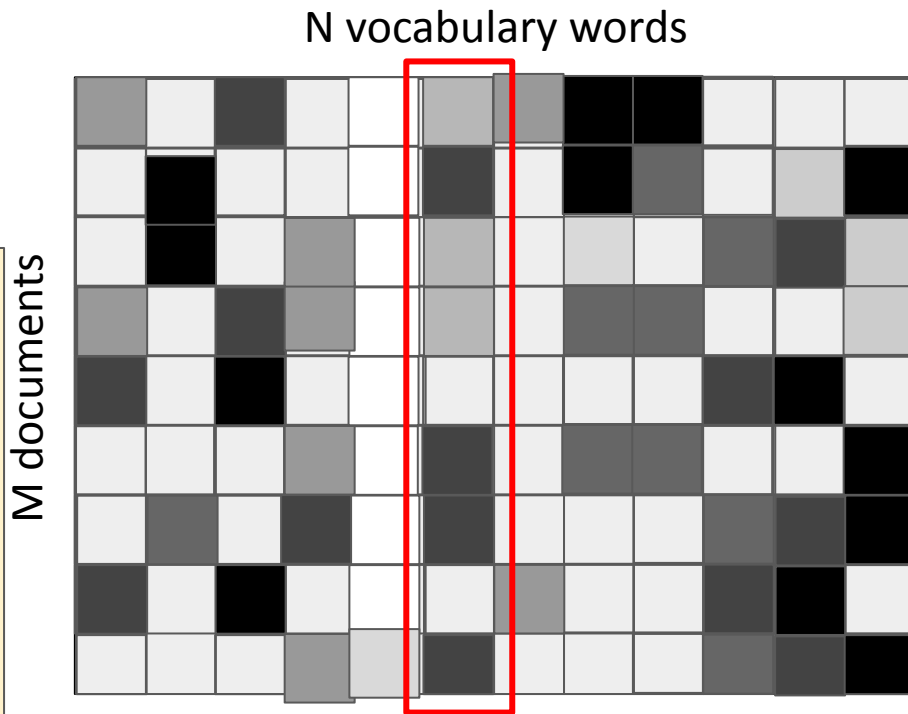


Last week...

Term-document matrix: a matrix of per-document word frequencies (counts)

$$T =$$

The j th column of T (or, the j th row of the *transposed* matrix T^T) is a **vector representation** of the j th word.



Why does the term-document matrix work for IR?

(Empirically usually true) assumption: if two documents are related to each other, they will tend to contain a similar set of words.

Conversely: if two words are related to each other, they will tend to appear in a similar set of documents (i.e., they will tend to **co-occur**).

Why does the term-document matrix work for IR?

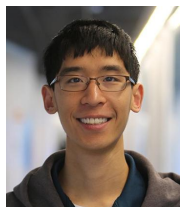
(Empirically usually true) assumption: if two documents are related to each other, they will tend to contain a similar set of words.

Conversely: if two words are related to each other, they will tend to appear in a similar set of documents (i.e., they will tend to **co-occur**).

“You shall know a word by the company it keeps.”

Why does(n't) the term-document matrix work for IR?

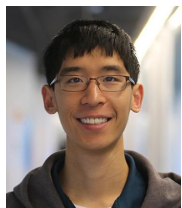
Hey Prof Chang, check out the recipe review finder app we made for our CS159 final project!



Wow, looks cool! I will test it by searching for “**cold breakfast**”.

Why does(n't) the term-document matrix work for IR?

Hey Prof Chang, check out the recipe review finder app we made for our CS159 final project!



Wow, looks cool! I will test it by searching for “**cold breakfast**”.



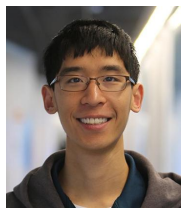
1: “The frozen peanut butter smoothie bowl is a perfect start to your morning”



2: “This ramen is a great choice for a cold Fall evening”

Why does(n't) the term-document matrix work for IR?

Hey Prof Chang, check out the recipe review finder app we made for our CS159 final project!



Wow, looks cool! I will test it by searching for **“cold breakfast”**.



1: “The frozen peanut butter smoothie bowl is a perfect start to your morning”



2: “This ramen is a great choice for a cold Fall evening”

DISCUSS: Using only cosine similarity on a term-document matrix (like we learned about last week), which document is more similar to the query “cold breakfast”?

Why does(n't) the term-document matrix work for IR?

On average, related words like “cold” and “frozen” will *tend* to co-occur.

This does not guarantee that *every* document containing “cold” will also contain “frozen”.

But if we know (through co-occurrence) that “cold” and “frozen” are related words, can't we leverage that to improve our IR?

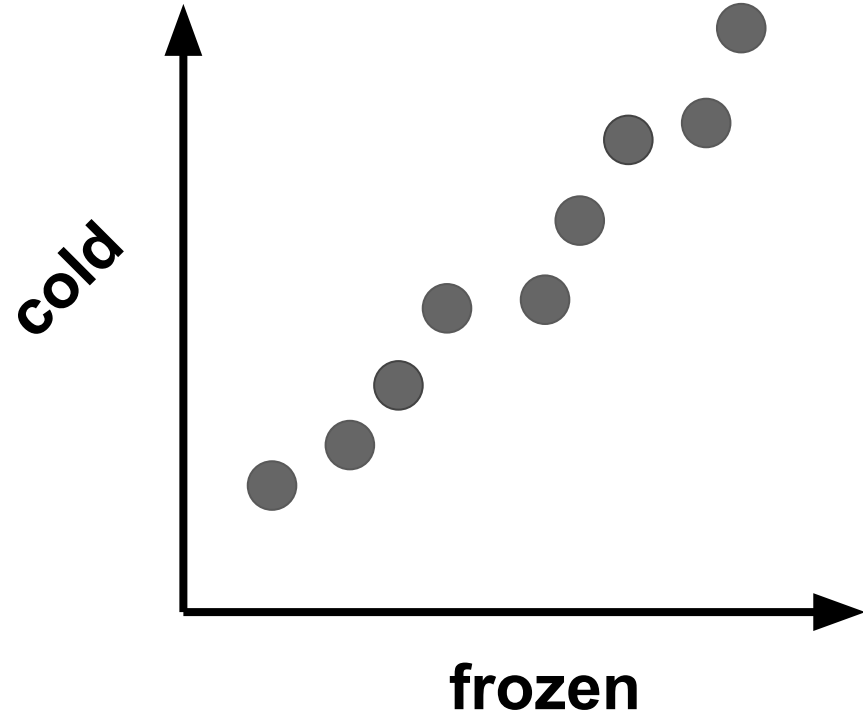
Key concept: **dimensionality** of data

Key concept: **dimensionality** of data

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5

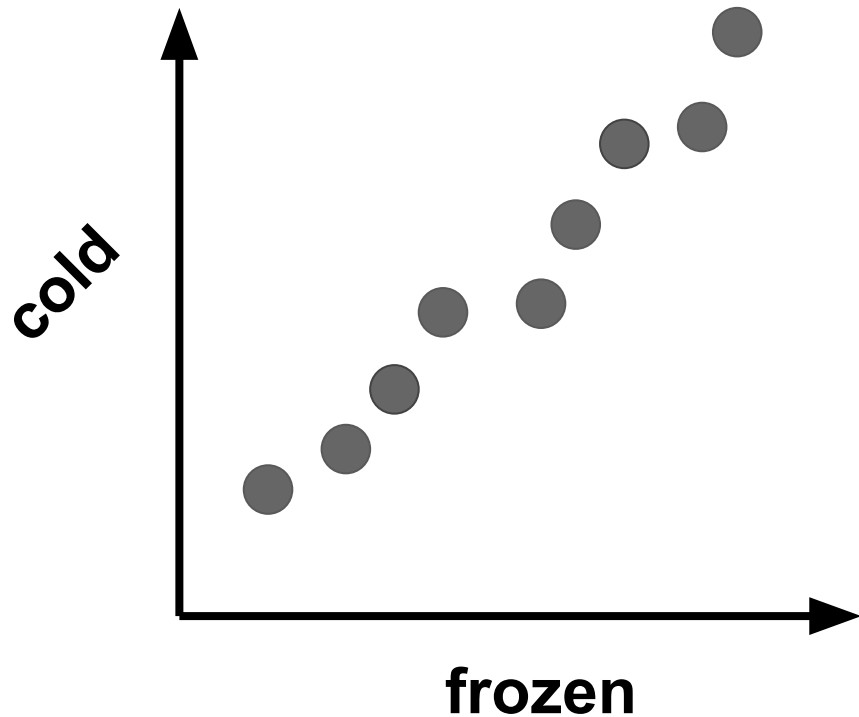
Key concept: **dimensionality** of data

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5



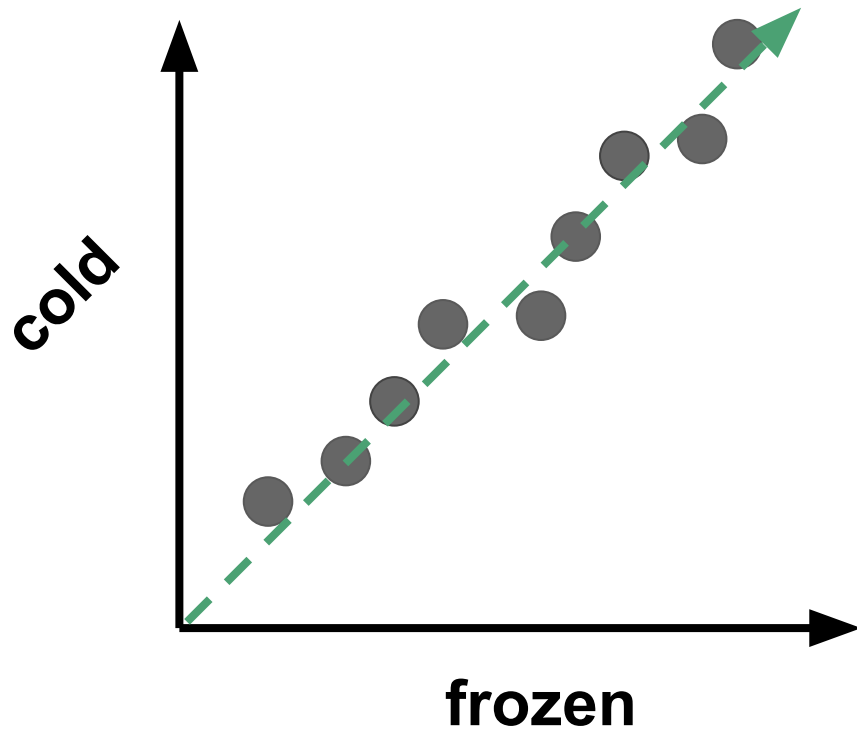
How many dimensions do we actually need?

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5



How many dimensions do we actually need?

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5



How many dimensions do we actually need?

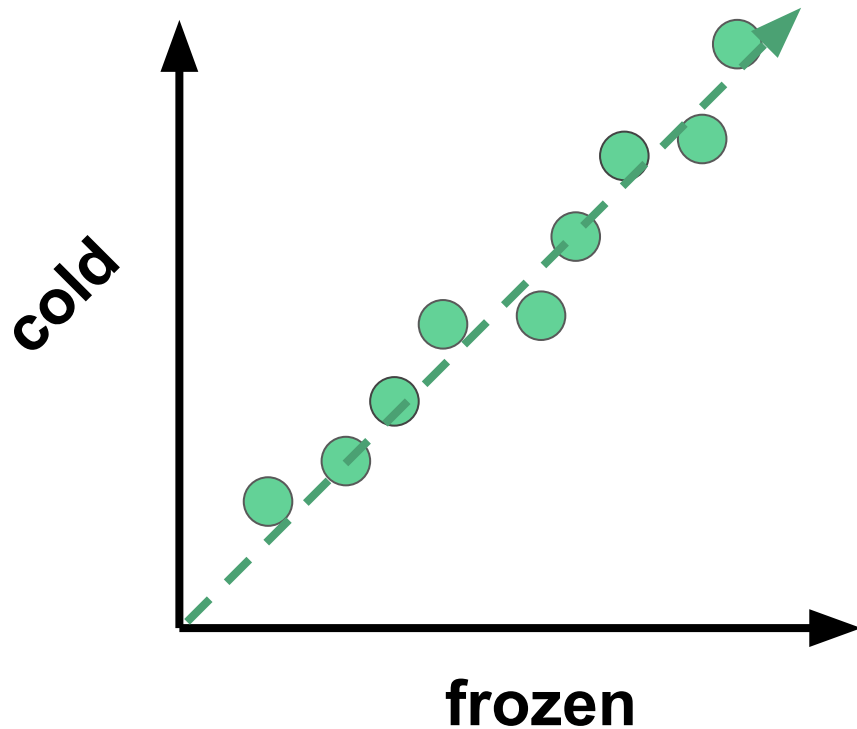
	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5

(redundant information)

2x	1x
----	----



5
2
2
0
9
3
8
2
5



How many dimensions do we actually need?

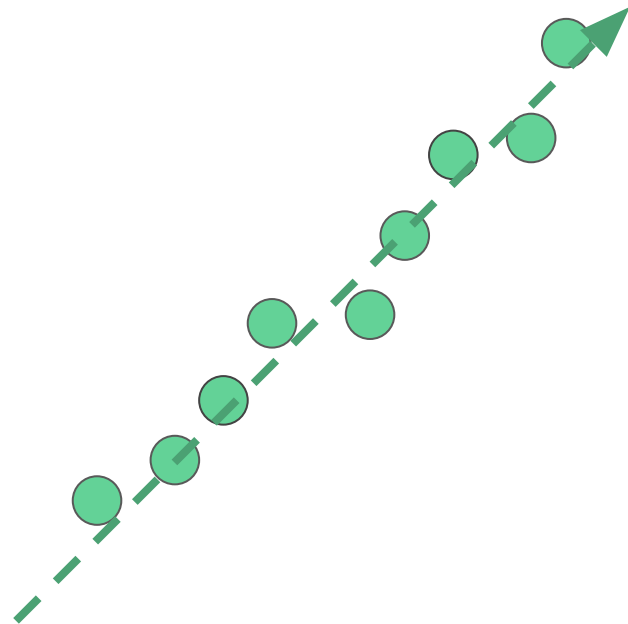
	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5

(redundant information)

2x	1x
----	----



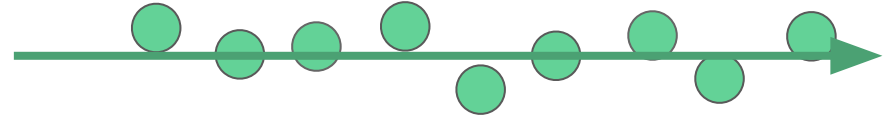
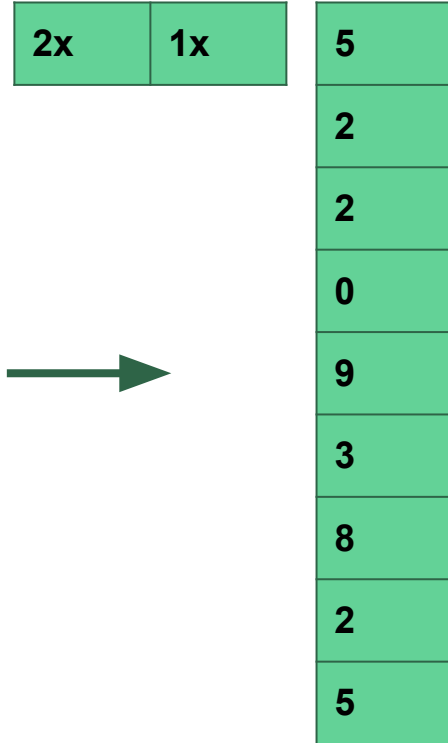
5
2
2
0
9
3
8
2
5



One Dimension!

How many dimensions do we actually need?

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5



1 NEW Dimension!

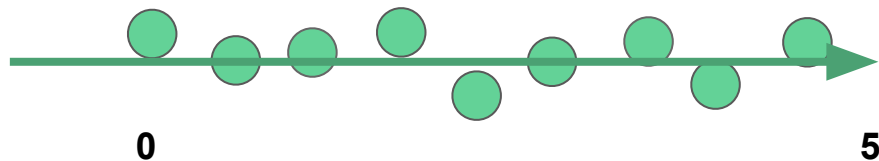
How many dimensions do we actually need?

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5

4x	2x
----	----



2.5
1
1
0
4.5
1.5
4
1
2.5

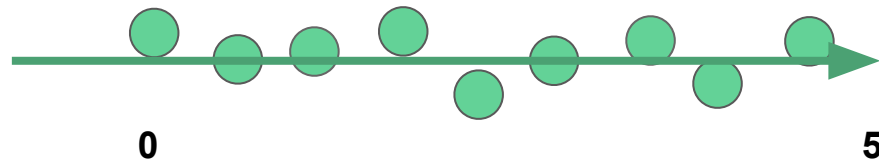


1 NEW Dimension!

How many dimensions do we actually need?

	cold	frozen
d1	10	5
d2	4	2
d3	3	2
d4	0	0
d5	20	9
d6	6	3
d7	17	8
d8	4	2
d9	8	5

4x	2x
2.5	
1	
1	
0	
4.5	
1.5	
4	
1	
2.5	



1 LATENT Dimension!
("coldness")

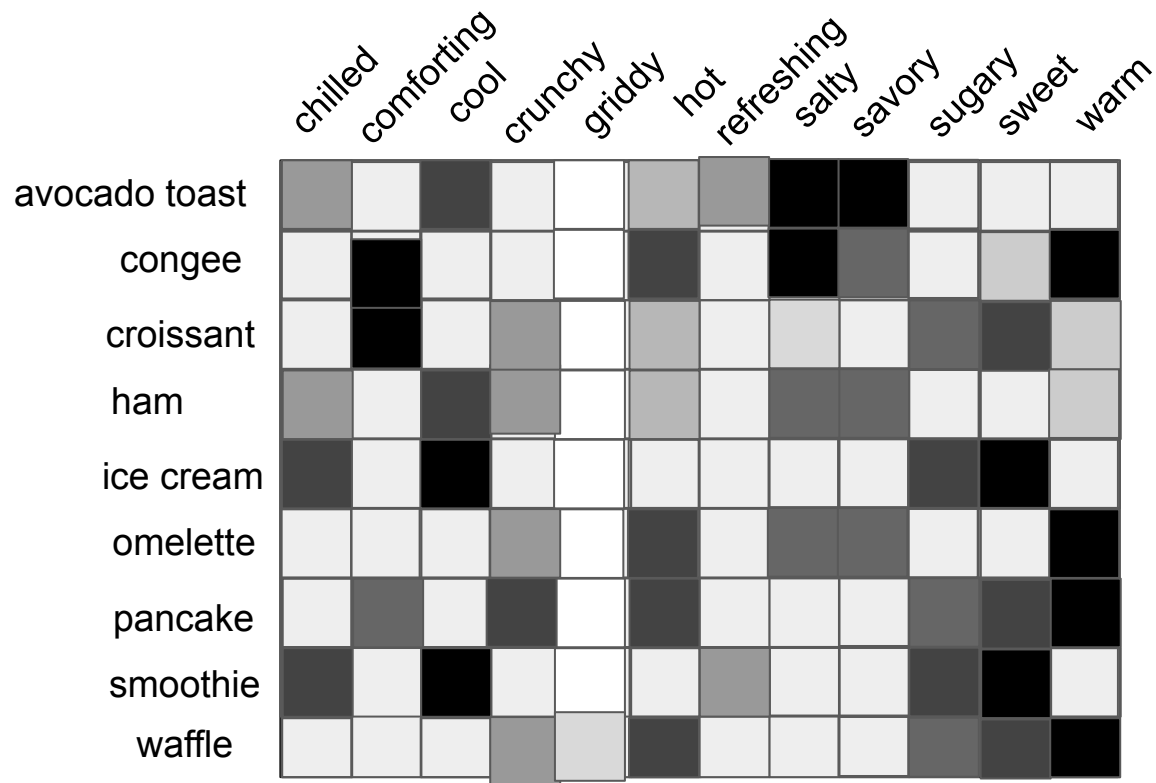
In-class activity:

what are the key dimensions of recipes?

Avocado toast is a savory, nutritious breakfast food consisting of avocados and bread. It can be served cool...

Congee is a comforting food, typically served at breakfast and generally savory. Think of hot soup but way better...

Pancakes are a sugary and delicious breakfast, and taste especially decadent when fresh and hot off of the pan...



**sweet vs
savory**

**warm vs
cold**

Solid vs liquid

avocado toast

congee

croissant

ham

ice cream

omelette

pancake

smoothie

waffle

These dimensions are **complementary** to each other and give a **complete** representation of the original matrix...

	sweet vs savory	warm vs cold	solid vs liquid
avocado toast	blue	purple	pink
congee	purple	red	blue
croissant	pink	purple	red
ham	blue	purple	red
ice cream	red	blue	purple
omelette	blue	red	pink
pancake	pink	pink	red
smoothie	red	blue	blue
waffle	pink	pink	red

These dimensions are **complementary** to each other and give a **complete** representation of the original matrix...

	sweet vs savory	warm vs cold	solid vs liquid
avocado toast	blue	purple	pink
congee	purple	red	blue
croissant	pink	purple	red
ham	blue	purple	red
ice cream	red	blue	purple
omelette	blue	red	pink
pancake	pink	pink	red
smoothie	red	blue	blue
waffle	pink	pink	red

...and they also allow us to **recover** the original word frequencies.

	sweet vs savory	warm vs cold	solid vs liquid	
	blue	red	purple	comforting
	purple	red	purple	hot
	purple	blue	blue	refreshing
	red	purple	purple	sweet

These dimensions are **complementary** to each other and give a **complete** representation of the original matrix...

	sweet vs savory	warm vs cold	solid vs liquid
avocado toast	blue	purple	pink
congee	purple	red	blue
croissant	pink	purple	red
ham	blue	purple	red
ice cream	red	blue	purple
omelette	blue	red	pink
pancake	pink	pink	red
smoothie	red	blue	blue
waffle	pink	pink	red

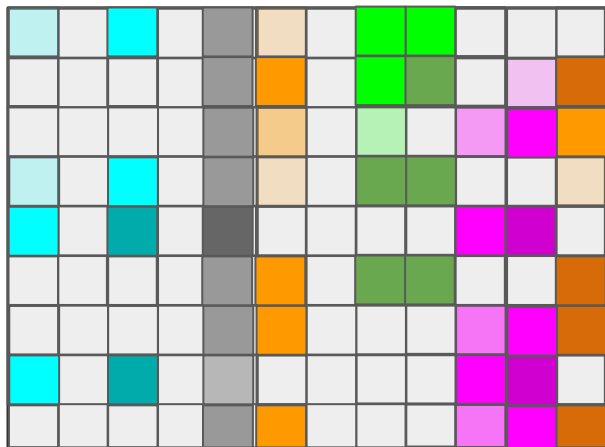
...and they also allow us to **recover** the original word frequencies.

sweet vs savory	warm vs cold	solid vs liquid	
blue	red	purple	comforting
purple	red	purple	hot
purple	blue	blue	refreshing
red	purple	purple	sweet

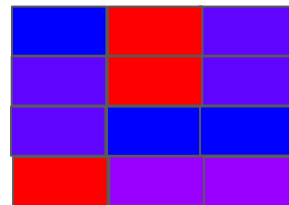
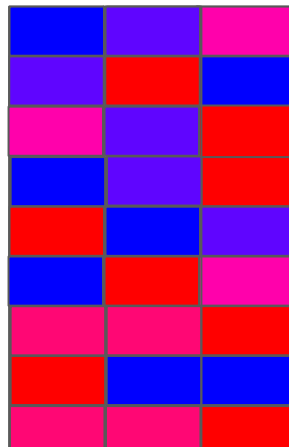
(notice that this also, conveniently, gives us a compact vector representation of each word!)

Recovering the original term-document matrix

T

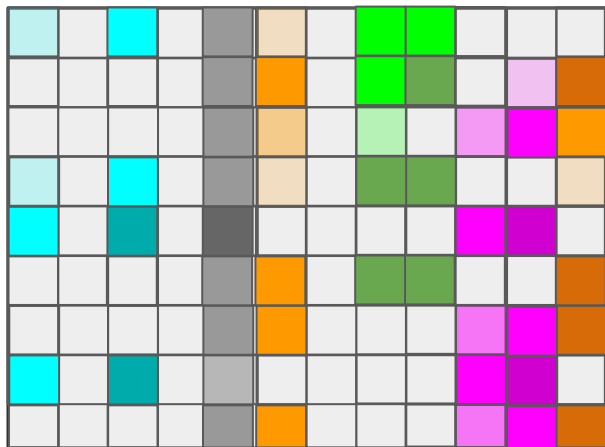


(term-document matrix)



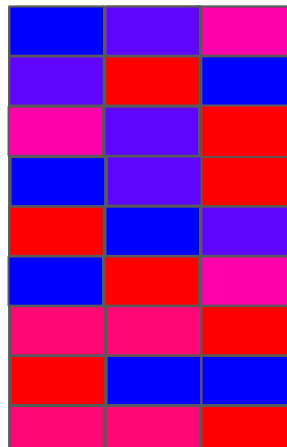
Recovering the original term-document matrix

T



(term-document matrix)

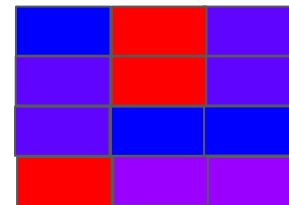
U



(document vectors)



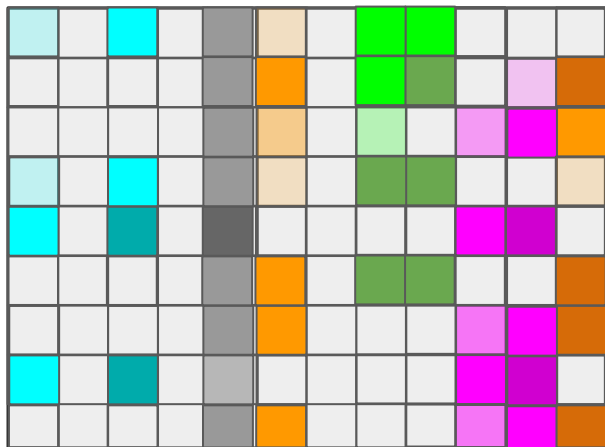
V



(word vectors)

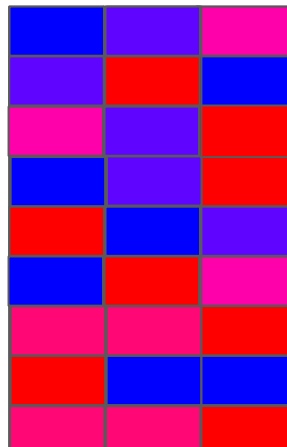
Recovering the original term-document matrix

T



(term-document matrix)

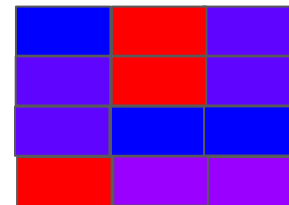
U



(document vectors)



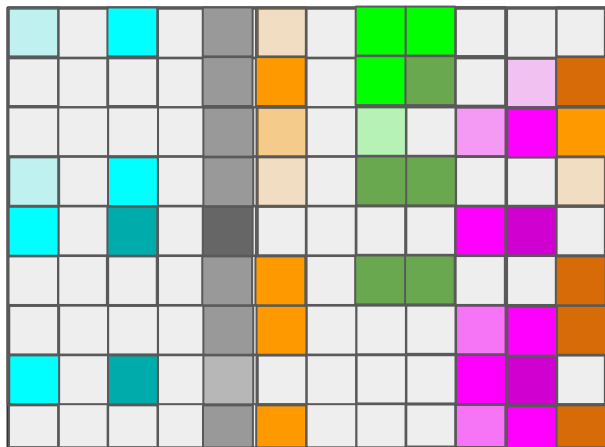
V



(word vectors)

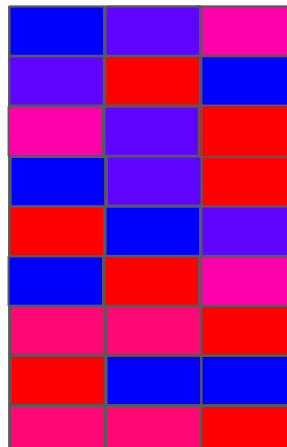
Recovering the original term-document matrix

T



(term-document matrix)

U

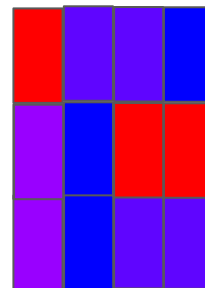


(document vectors)



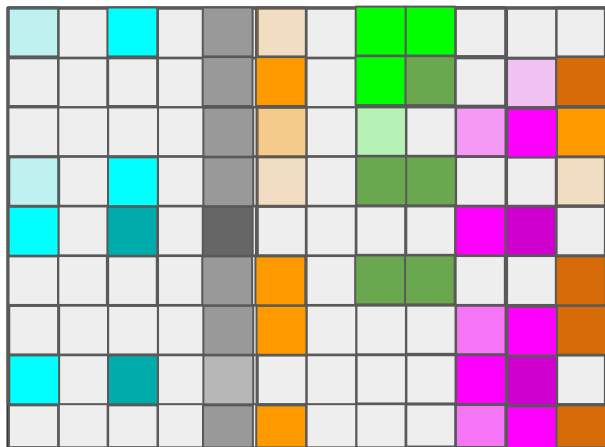
V^T

(word vectors)



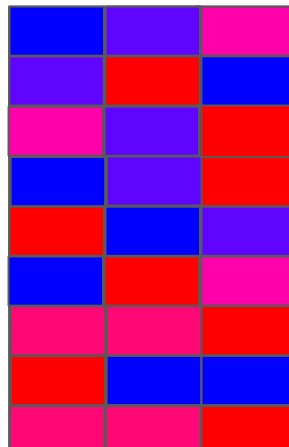
Recovering the original term-document matrix

T



(term-document matrix)

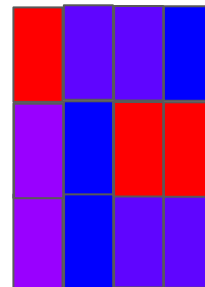
U



(document vectors)

V^T

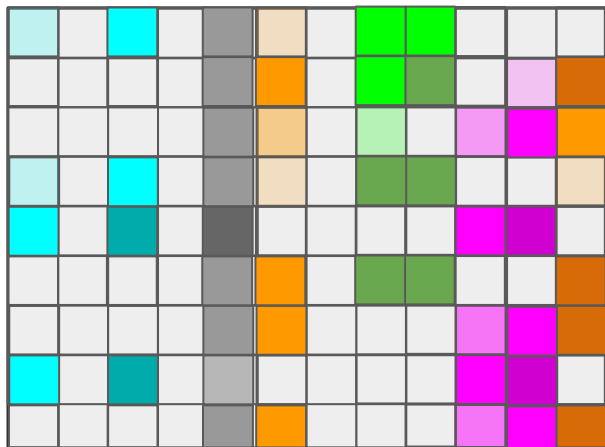
(word vectors)



$$T \approx UV^T$$

Recovering the original term-document matrix

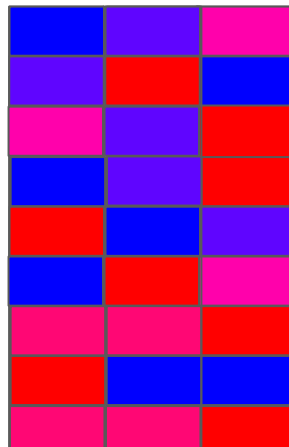
T



(term-document matrix)



U



(document vectors)

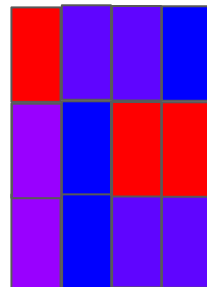
Σ



("singular values":
estimates of how
much of the data is
captured by each
dimension; think
of them like r^2
values)

V^T

(word vectors)



$$T \approx U \Sigma V^T$$

Finding U , Σ , and V^T

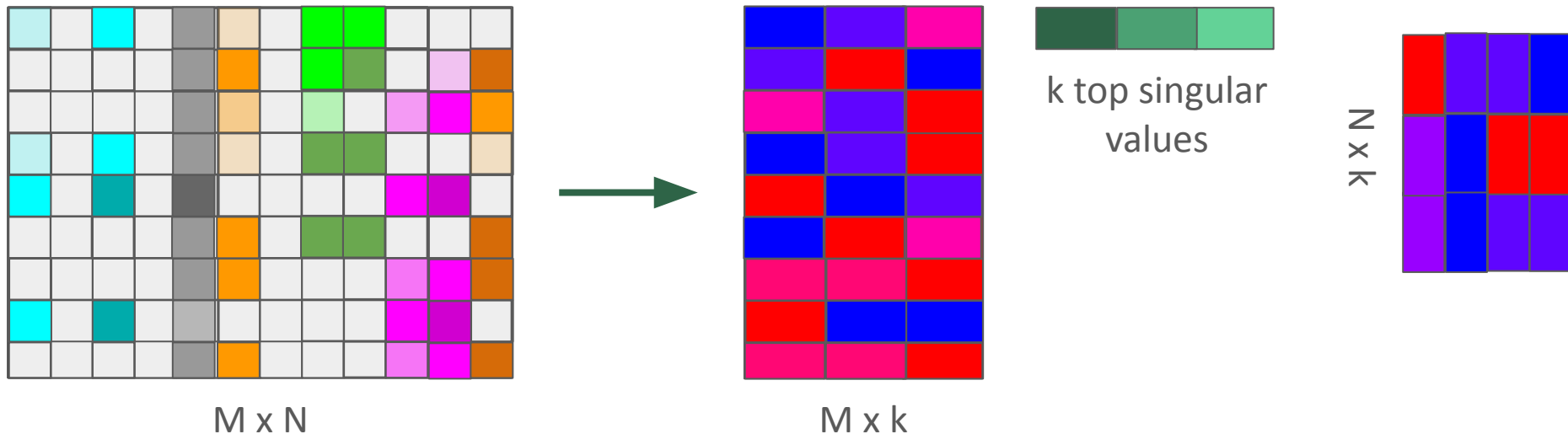
If the correlations were exact (no approximations), then $T = U\Sigma V^T$ could be *exactly* computed via **singular value decomposition** (SVD) (using eigenvector math)

Finding U , Σ , and V^T

For real (noisy) data, we instead use **truncated SVD**: keep only the top k dimensions (by order of singular values)

Finding U , Σ , and V^T

For real (noisy) data, we instead use **truncated SVD**: keep only the top k dimensions (by order of singular values)

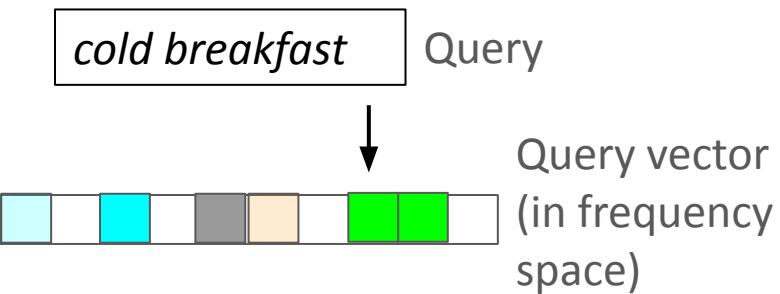


$$T \approx U_k \Sigma_k V_k^T$$

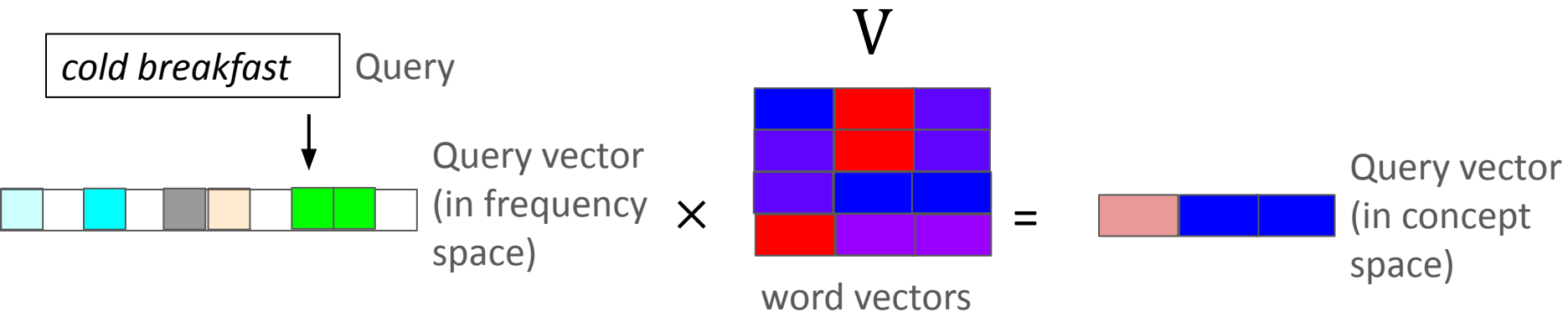
Applications of truncated SVD

Application #1: information retrieval that doesn't depend on exact word matches

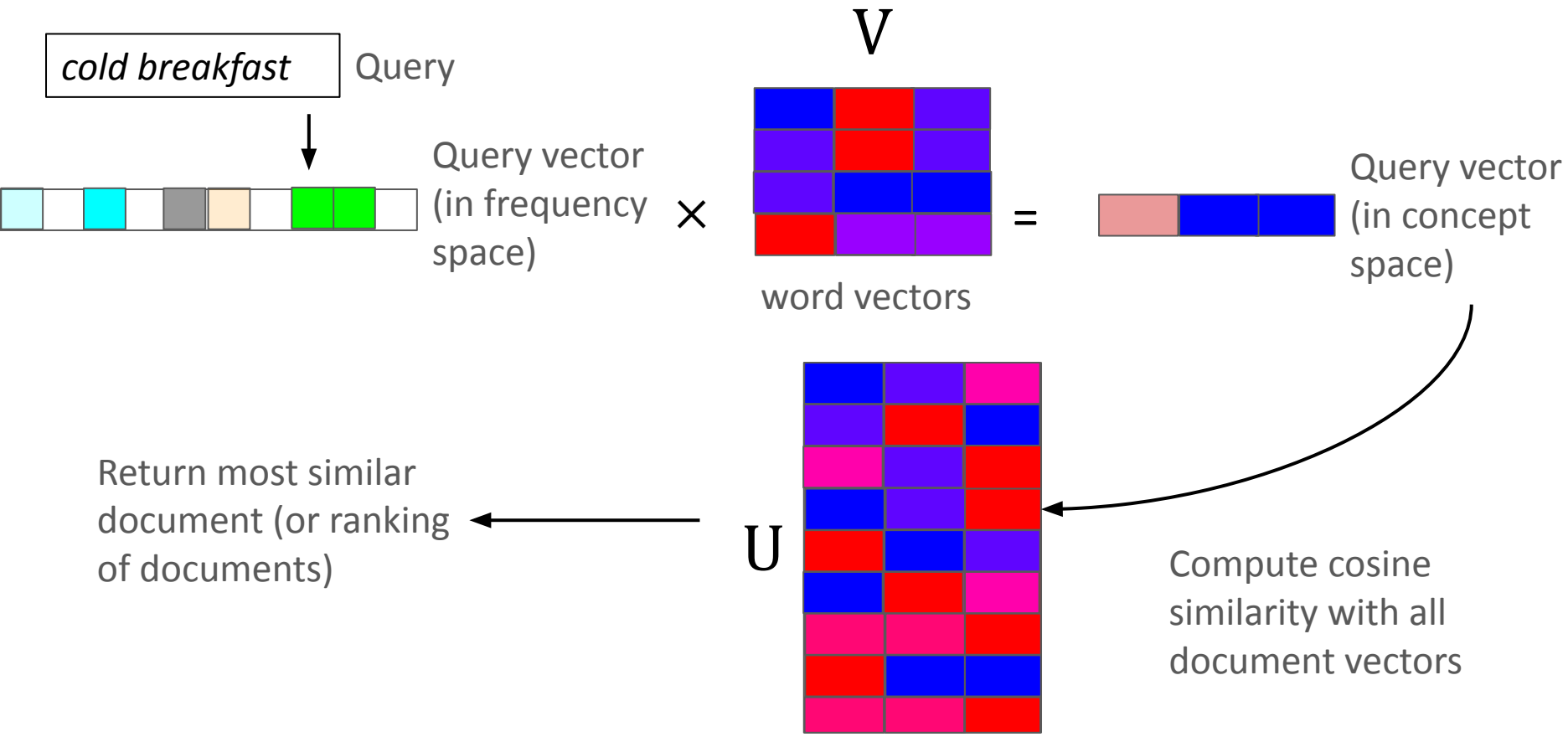
IR with truncated SVD



IR with truncated SVD



IR with truncated SVD



Applications of truncated SVD

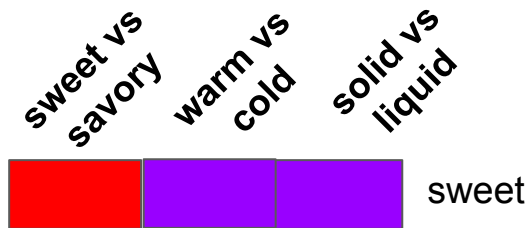
Application #1: information retrieval that doesn't depend on exact word matches

Application #2: compute **semantic similarity** of words based on concept overlap (we call this **vector semantics**)

Applications of truncated SVD

Application #1: information retrieval that doesn't depend on exact word matches

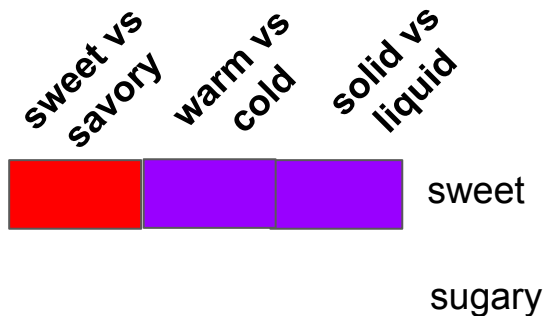
Application #2: compute **semantic similarity** of words based on concept overlap (we call this **vector semantics**)



Applications of truncated SVD

Application #1: information retrieval that doesn't depend on exact word matches

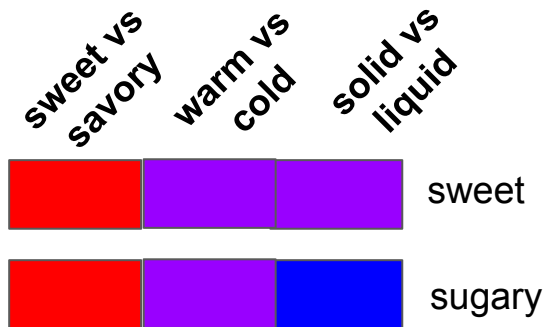
Application #2: compute **semantic similarity** of words based on concept overlap (we call this **vector semantics**)



Applications of truncated SVD

Application #1: information retrieval that doesn't depend on exact word matches

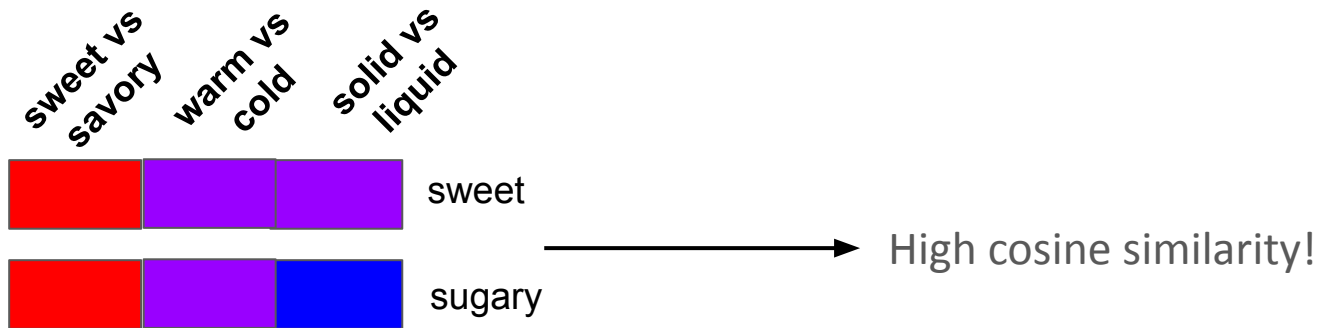
Application #2: compute **semantic similarity** of words based on concept overlap (we call this **vector semantics**)



Applications of truncated SVD

Application #1: information retrieval that doesn't depend on exact word matches

Application #2: compute **semantic similarity** of words based on concept overlap (we call this **vector semantics**)

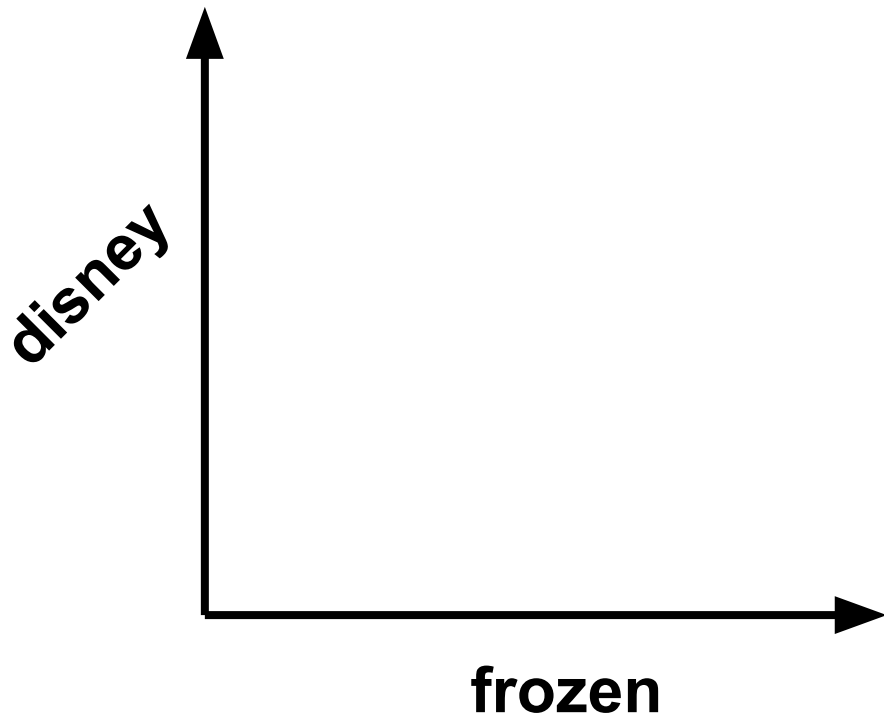


Limitations of SVD

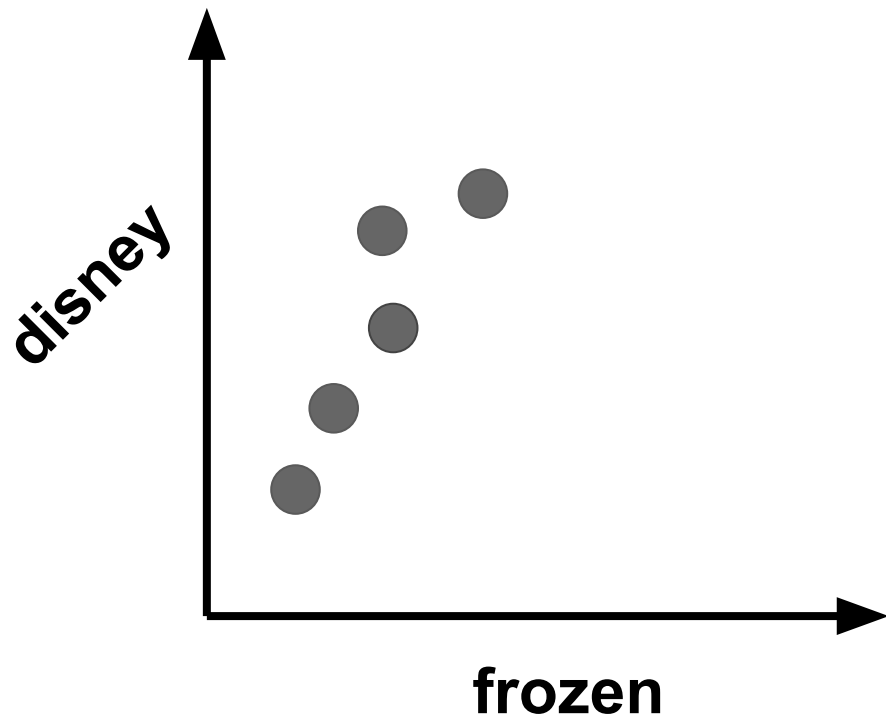
SVD is based on finding *linear* correlations between words (and solves for them using *linear* algebra)

But what if words are correlated *nonlinearly*?

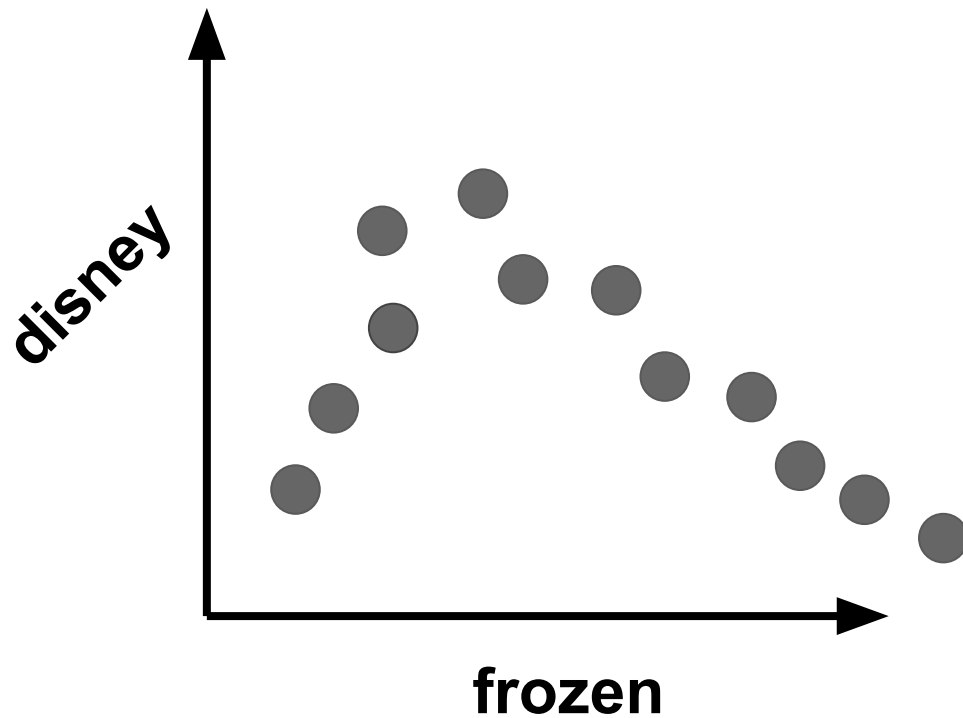
Addressing nonlinear correlations



Addressing nonlinear correlations



Addressing nonlinear correlations



Addressing nonlinear correlations

Two complications arise when considering nonlinear correlations:

1. More than two matrices may be needed $T \approx U_1 ?? U_2 ?? \dots ?? U_L$
2. We can no longer rely on *linear* algebra to find those matrices

Addressing nonlinear correlations

Two complications arise when considering nonlinear correlations:

1. More than two matrices may be needed $T \approx U_1 ?? U_2 ?? \dots ?? U_L$
2. We can no longer rely on *linear* algebra to find those matrices

Instead, we model T as a nonlinear system of matrix transformations known as a **neural network** (for more, take CS152)

Addressing nonlinear correlations

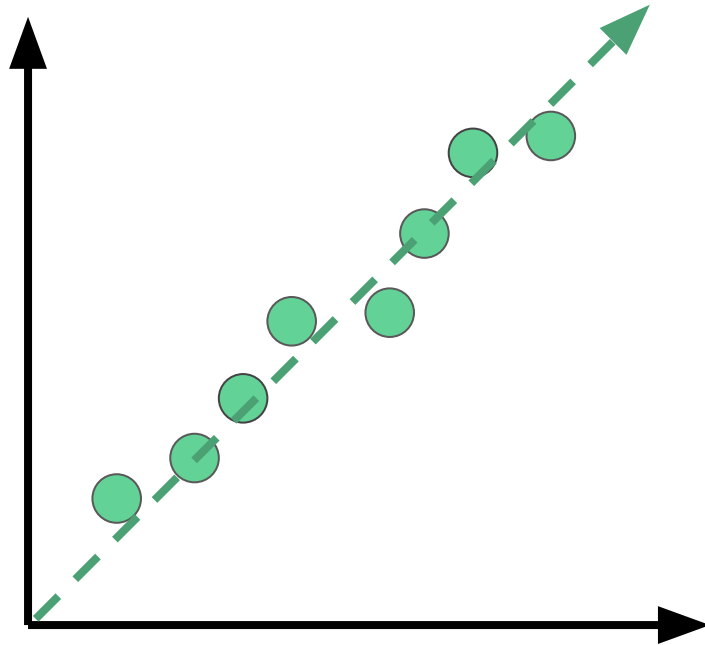
Two complications arise when considering nonlinear correlations:

1. More than two matrices may be needed $T \approx U_1 ?? U_2 ?? \dots ?? U_L$
2. We can no longer rely on *linear* algebra to find those matrices

Instead, we model T as a nonlinear system of matrix transformations known as a **neural network** (for more, take CS152)

Neural networks have shown effectiveness at creating word vectors (e.g., word2vec by Mikolov et al., 2013) *or* creating document vectors (e.g., doc2vec by Le and Mikolov, 2014)

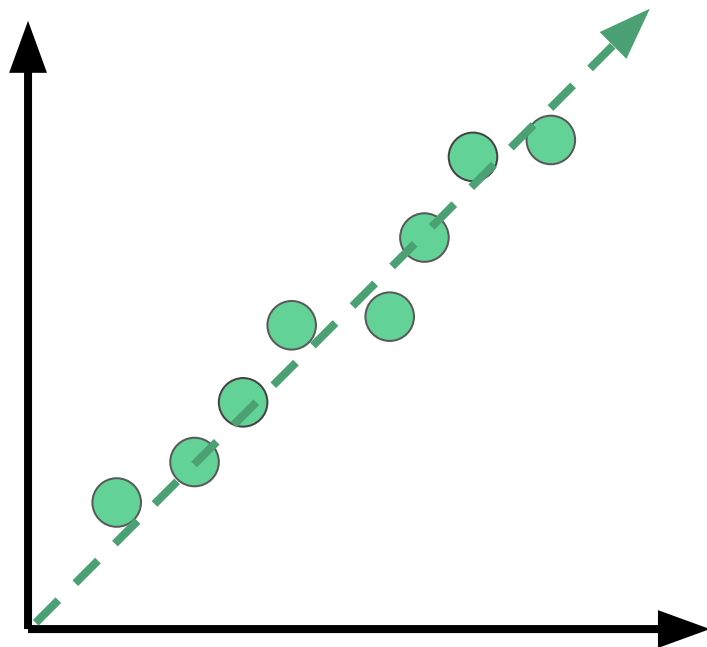
Two sides of the same coin...



Correlation

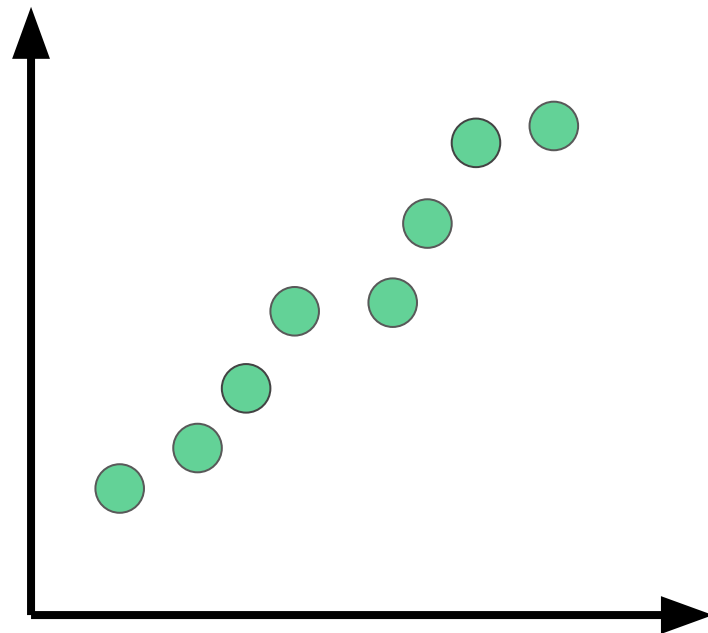
What is the **best explanation** for the observed data?

Two sides of the same coin...



Correlation

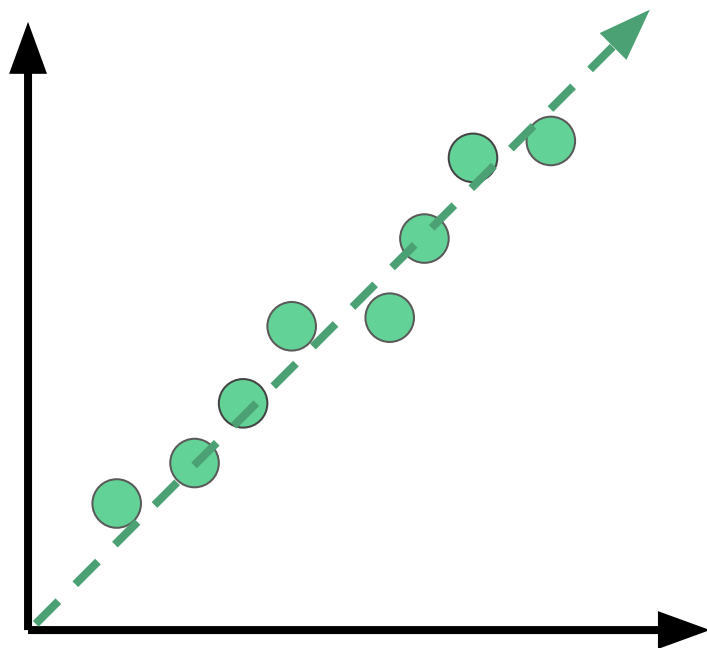
What is the **best explanation** for the observed data?



Prediction

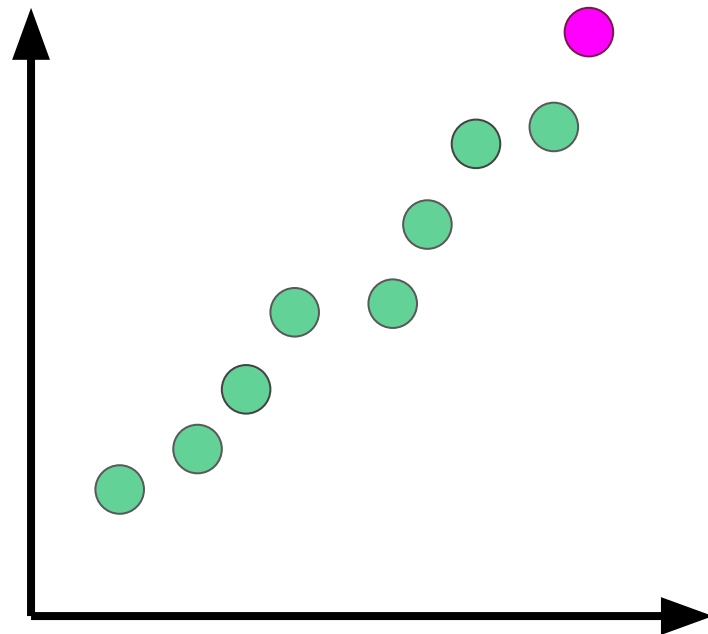
How to **best predict** unseen data given the observed data?

Two sides of the same coin...



Correlation

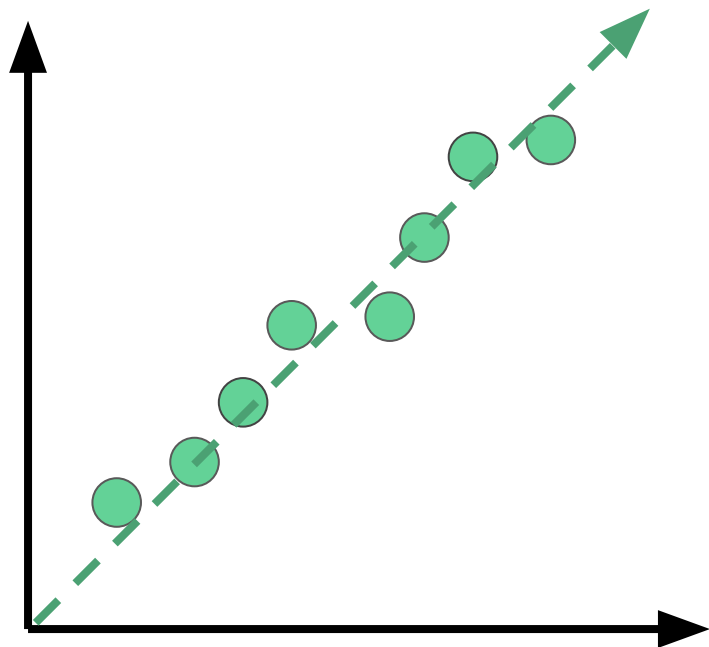
What is the **best explanation** for the observed data?



Prediction

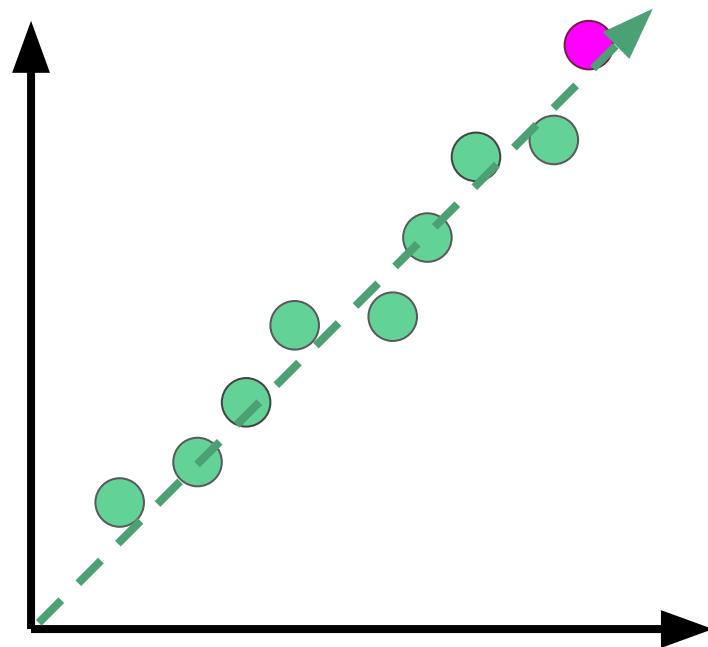
How to **best predict** unseen data given the observed data?

Two sides of the same coin...



Correlation

What is the **best explanation** for the observed data?



Prediction

How to **best predict** unseen data given the observed data?

Have a great Fall Break!

Please come to class after break (Thursday Oct 16) for a **special lecture** that connects everything we've learned to modern LLMs!